NAME: S. Venkata Praveen

Reg no: 192373013

Course : Theory of computations (CSA1303)

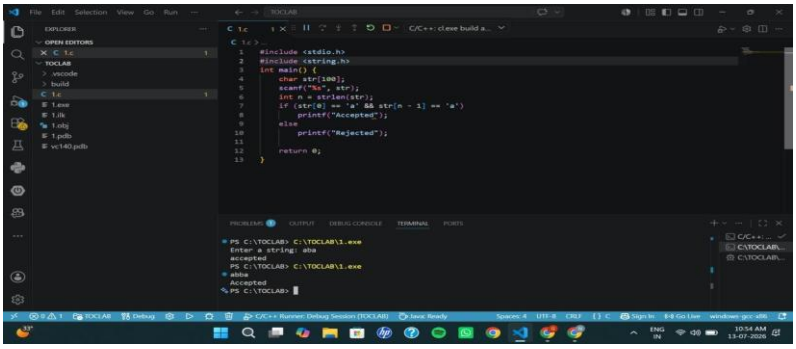
Exercise 1

Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a

**Code:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    scanf("%s", str);
    int n = strlen(str);
    if (str[0] == 'a' && str[n - 1] == 'a')
        printf("Accepted");
    else
        printf("Rejected");
    return 0;
}
```

**Output:**



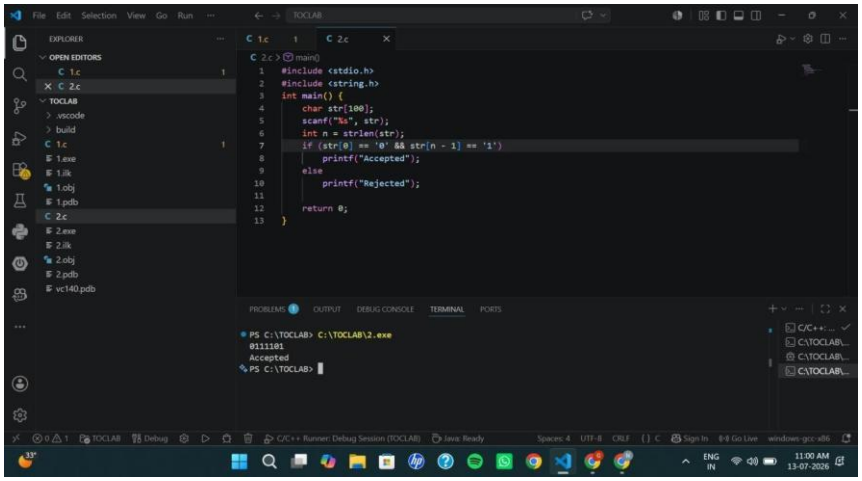
Exercise 2

Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with 0 and end with 1

**Code:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    scanf("%s", str);
    int n = strlen(str);
    if (str[0] == '0' && str[n - 1] == '1')
        printf("Accepted");
    else
        printf("Rejected");
    return 0;
}
```

**Output:**



Exercise 3

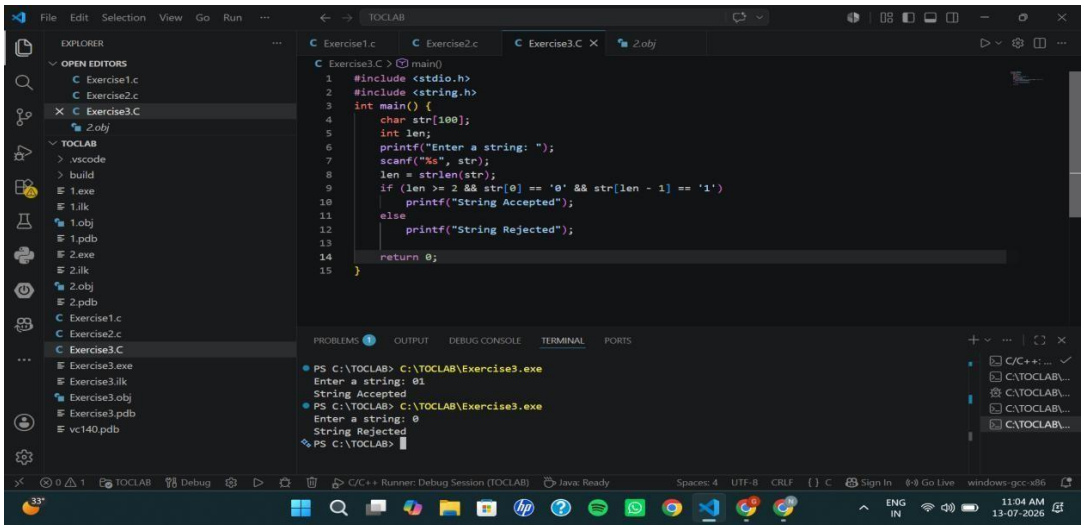
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

S → 0A1  A → 0A | 1A | ε

**Code:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int len;
    printf("Enter a string: ");
    scanf("%s", str);
    len = strlen(str);
    if (len >= 2 && str[0] == '0' && str[len - 1] == '1')
        printf("String Accepted");
    else
        printf("String Rejected");
    return 0;
}
```

**Output:**



Exercise 4

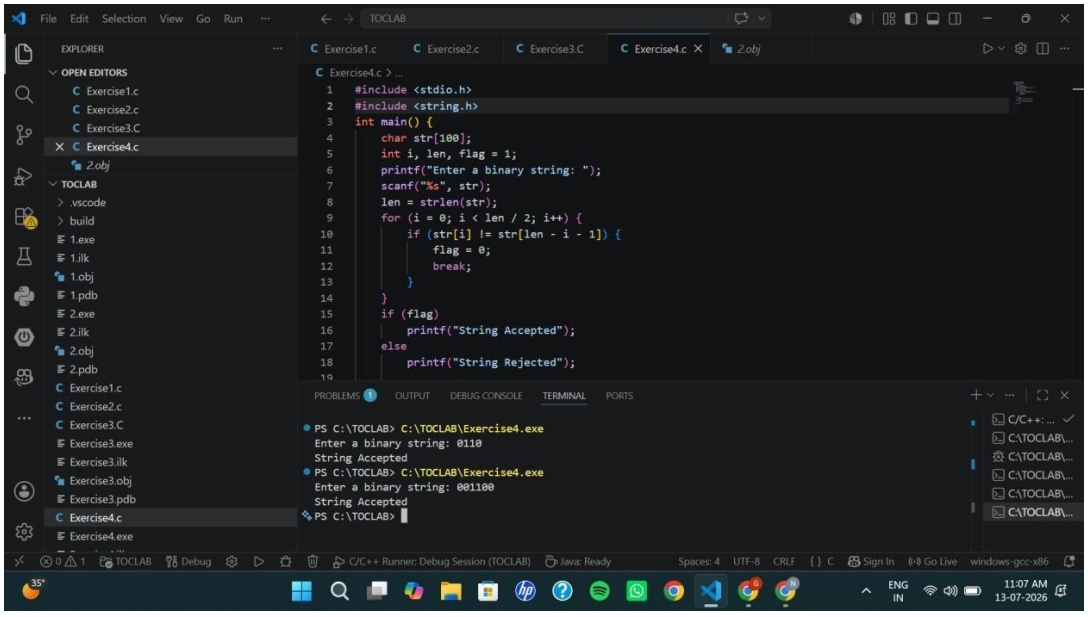
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

S → 0S0 | 1S1 | 0 | 1 | ε

**Code:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len, flag = 1;
    printf("Enter a binary string: ");
    scanf("%s", str);
    len = strlen(str);
    for (i = 0; i < len / 2; i++) {
        if (str[i] != str[len - i - 1]) {
            flag = 0;
            break;
        }
    }
    if (flag)
        printf("String Accepted");
    else
        printf("String Rejected");
    return 0;
}
```

**Output:**



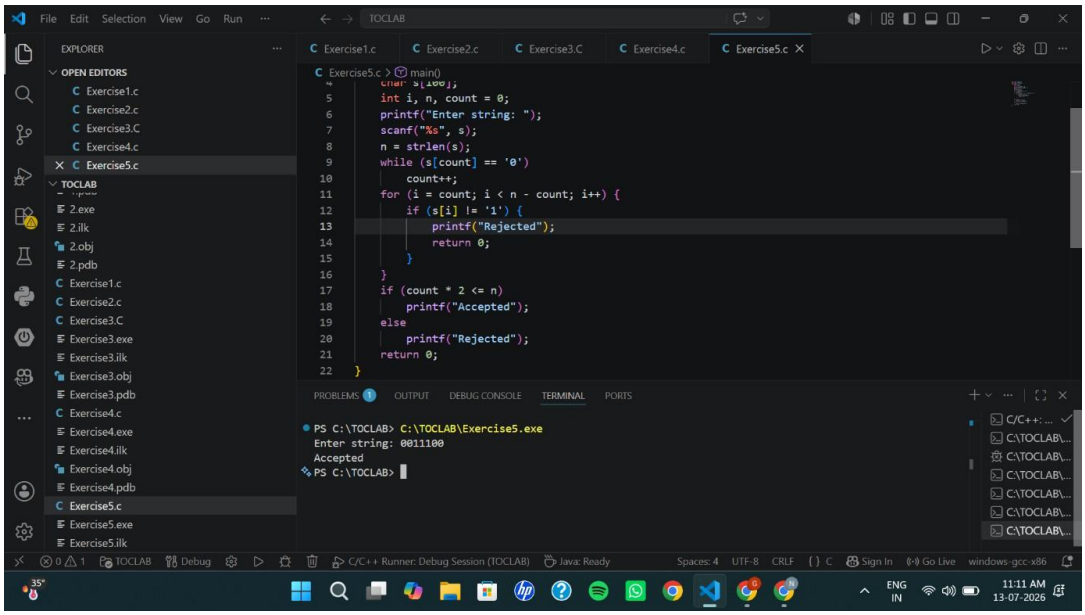
Exercise 5
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S0 \mid A \quad A \rightarrow 1A \mid \varepsilon$

**Code:**

```
#include <stdio.h>
#include <string.h>
int main() {
   char s[100];
   int i, n, count = 0;
   printf("Enter string: ");
   scanf("%s", s);
   n = strlen(s);
   while (s[count] == '0')
      count++;
   for (i = count; i < n - count; i++) {
      if (s[i] != '1') {
         printf("Rejected");
         return 0;
      }
   }
   if (count * 2 <= n)
      printf("Accepted");
   else
      printf("Rejected");
   return 0;
}
```

**Output:**


File Edit Selection View Go Run
TOCLAB
EXPLORER
OPEN EDITORS
Exercise1.c
Exercise2.c
Exercise3.C
Exercise4.c
Exercise5.c
TOCLAB
2.exe
2.ilk
2.obj
2.pdb
Exercise1.c
Exercise2.c
Exercise3.C
Exercise3.exe
Exercise3.ilk
Exercise3.obj
Exercise3.pdb
Exercise4.c
Exercise4.exe
Exercise4.ilk
Exercise4.obj
Exercise4.pdb
Exercise5.c
Exercise5.exe
Exercise5.ilk
Exercise5.c > main()
int i, n, count = 0;
printf("Enter string: ");
scanf("%s", s);
n = strlen(s);
while (s[count] == '0')
count++;
for (i = count; i < n - count; i++) {
if (s[i] != '1') {
printf("Rejected");
return 0;
}
}
if (count * 2 <= n)
printf("Accepted");
else
printf("Rejected");
return 0;
}
PROBLEMS 1
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\TOCLAB> C:\TOCLAB\Exercise5.exe
Enter string: 0011100
Accepted
PS C:\TOCLAB>
Debug
C/C++ Runner: Debug Session (TOCLAB)
Java: Ready
Spaces: 4
UTF-8
CRLF
C
Sign In
Go Live
windows-gcc-x86
ENG
IN
11:11 AM
13-07-2026

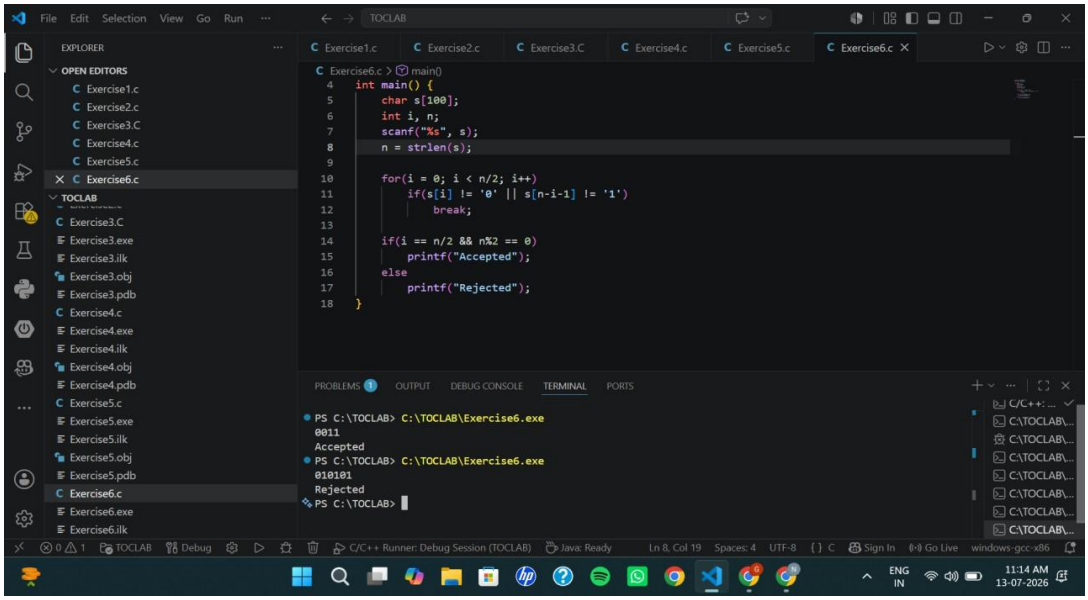
Exercise 6
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

S → 0S1 | ε

**Code:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i, n;
    scanf("%s", s);
    n = strlen(s);
    for(i = 0; i < n/2; i++)
        if(s[i] != '0' || s[n-i-1] != '1')
            break;
    if(i == n/2 && n%2 == 0)
        printf("Accepted");
    else
        printf("Rejected");
}
```

**Output:**



Exercise 7

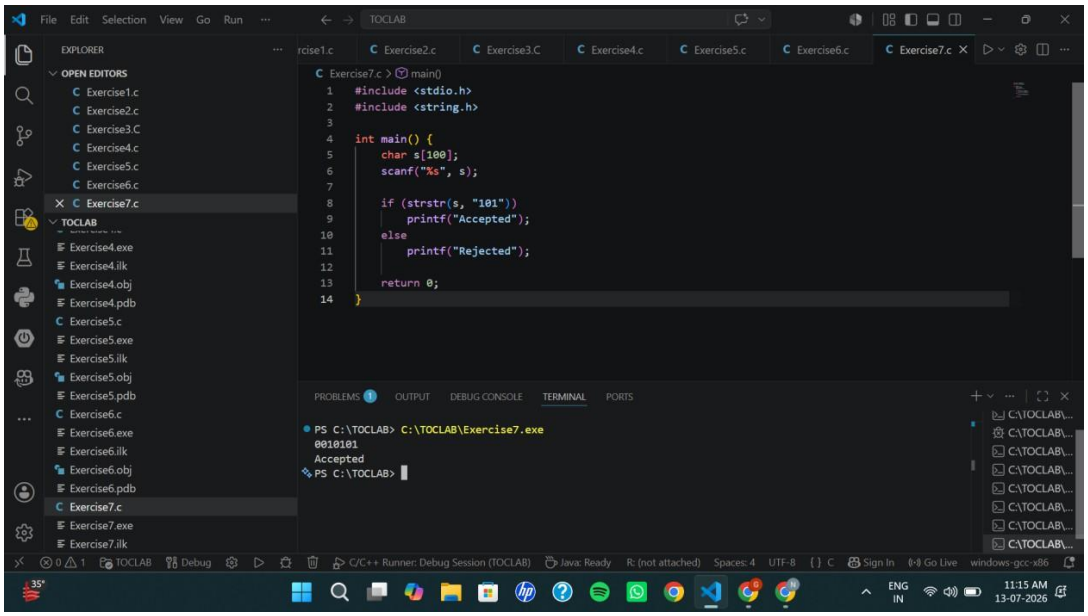
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

S → A101A, A → 0A | 1A | ε

**Code:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    scanf("%s", s);
    if (strstr(s, "101"))
        printf("Accepted");
    else
        printf("Rejected");
    return 0;
}
```

**Output:**

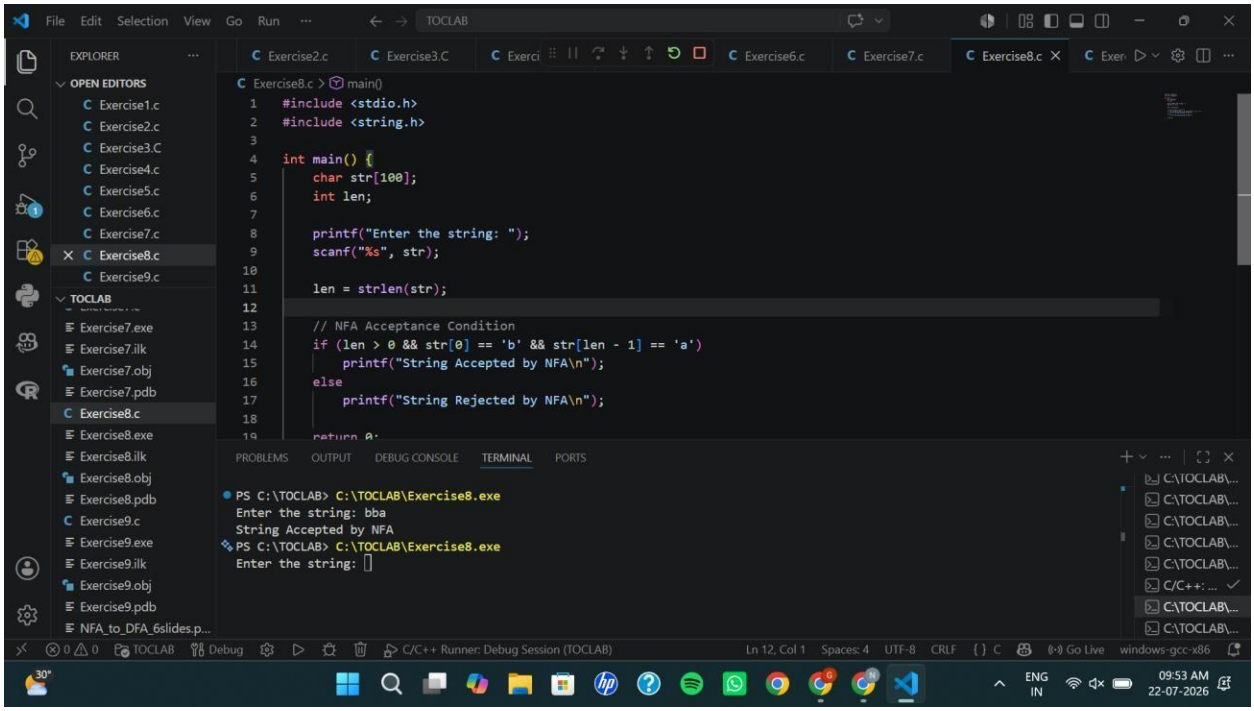


Exercise 8:
Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given languagerepresenting strings that start with b and end with a

**Code:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int len;

    printf("Enter the string: ");
    scanf("%s", str);

    len = strlen(str);

    // NFA Acceptance Condition
    if (len > 0 && str[0] == 'b' && str[len - 1] == 'a')
        printf("String Accepted by NFA\n");
    else
        printf("String Rejected by NFA\n");

    return 0;
}
```

**Output:**



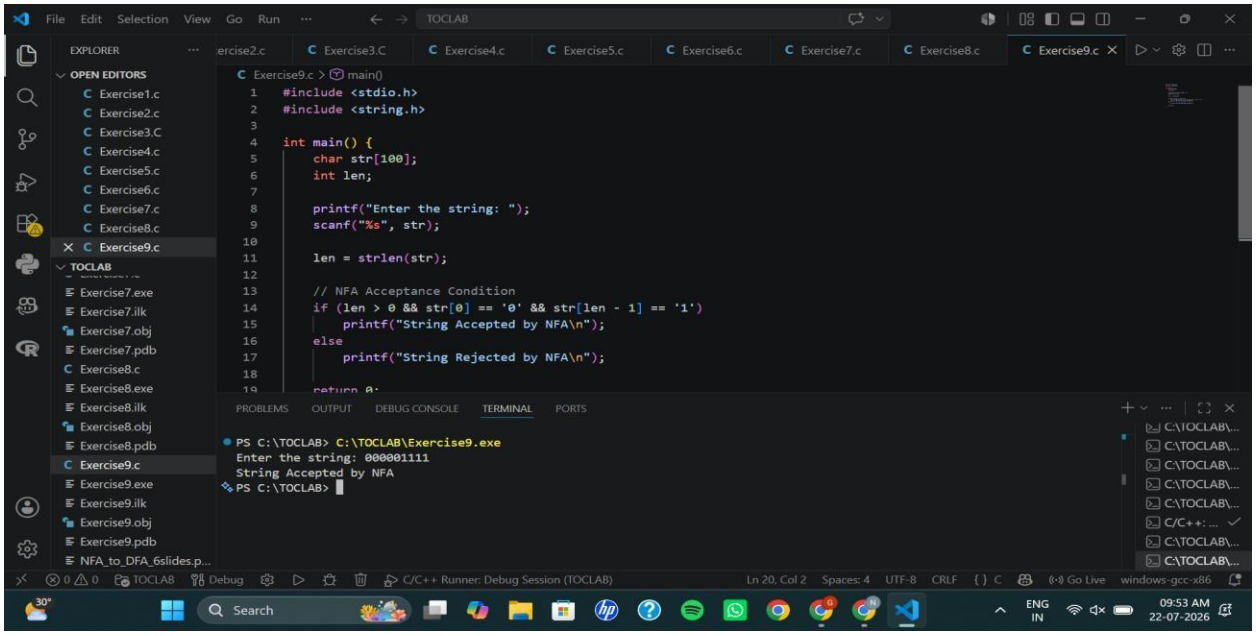
Exercise 9

Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given languagerepresenting strings that start with o and end with 1

**Code:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int len;

    printf("Enter the string: ");
    scanf("%s", str);

    len = strlen(str);

    // NFA Acceptance Condition
    if (len > 0 && str[0] == '0' && str[len - 1] == '1')
        printf("String Accepted by NFA\n");
    else
        printf("String Rejected by NFA\n");

    return 0;
}
```

**Output:**



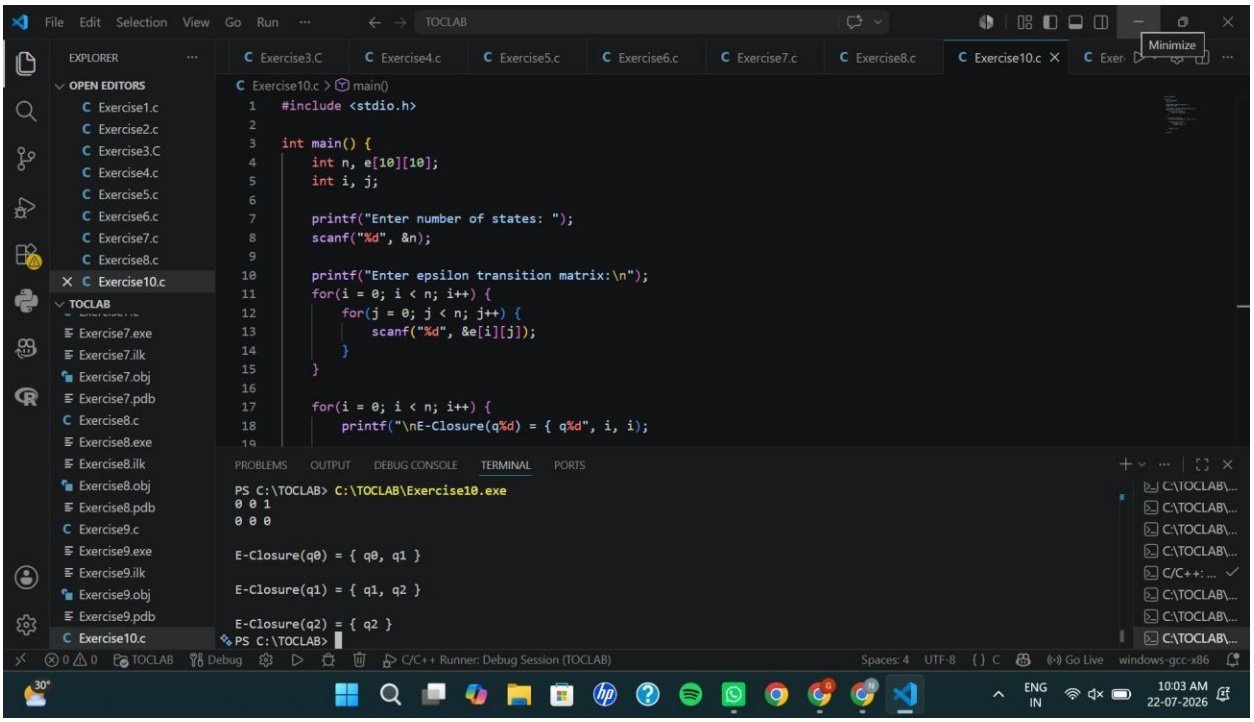
Exercise 10

Write a C program to find ε -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ε -moves.

**Code:**

```
#include <stdio.h>
int main() {
    int n, e[10][10];
    int i, j;
    printf("Enter number of states: ");
    scanf("%d", &n);
    printf("Enter epsilon transition matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &e[i][j]);
        }
    }
    for(i = 0; i < n; i++) {
        printf("\nE-Closure(q%d) = { q%d", i, i);
        for(j = 0; j < n; j++) {
            if(e[i][j] == 1)
                printf(", q%d", j);
        }
        printf(" }\n");
    }
    return 0;
}
```

**Output:**

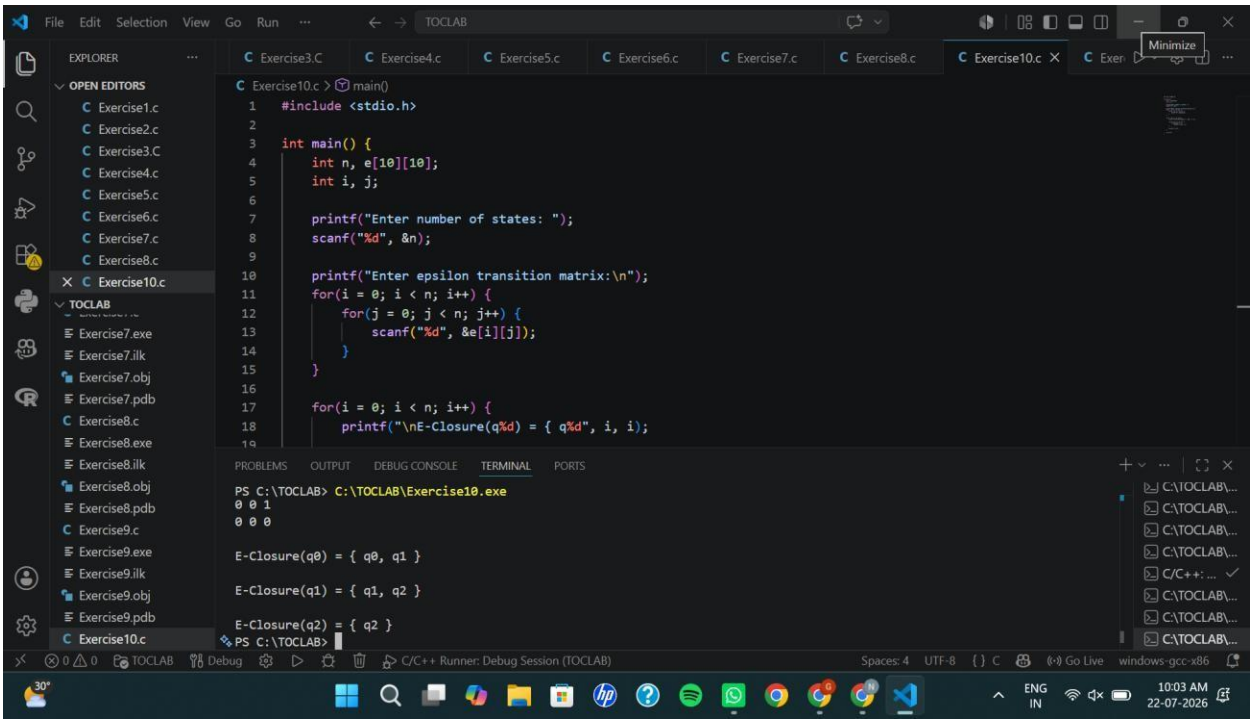


Exercise 11:
Write a C program to find ε -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ε -moves.

**Code:**

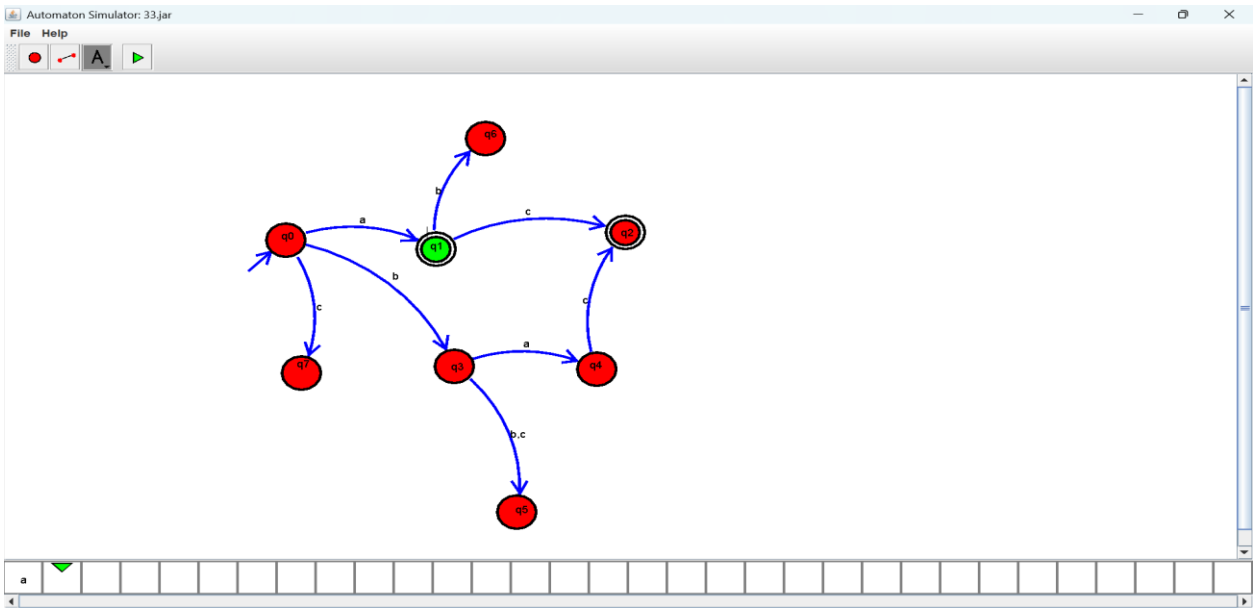
```
#include <stdio.h>
int main() {
   int n, e[10][10];
   int i, j;
   printf("Enter number of states: ");
   scanf("%d", &n);
   printf("Enter epsilon transition matrix:\n");
   for(i = 0; i < n; i++) {
      for(j = 0; j < n; j++) {
         scanf("%d", &e[i][j]);
      }
   }
   for(i = 0; i < n; i++) {
      printf("\nE-Closure(q%d) = { q%d", i, i);
      for(j = 0; j < n; j++) {
         if(e[i][j] == 1)
            printf(", q%d", j);
      }
      printf(" }\n");
   }
   return 0;
}
```

## Output



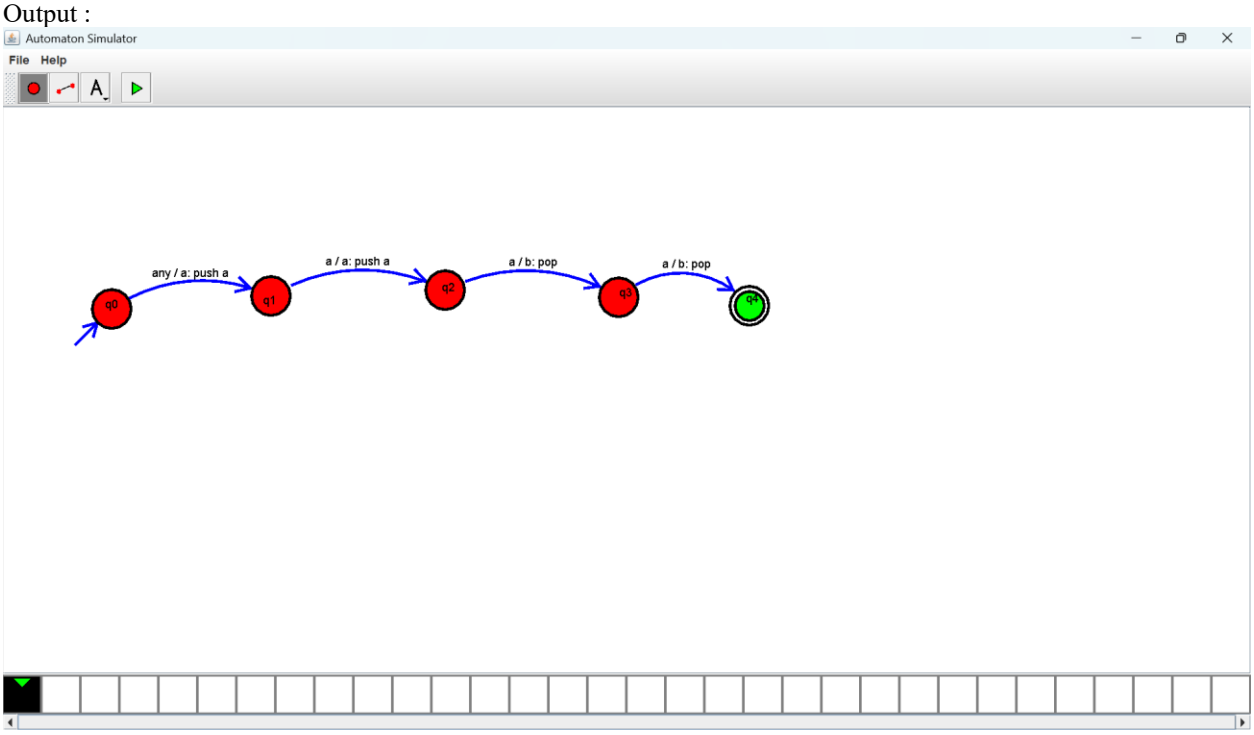

Exercise 12:

Output :



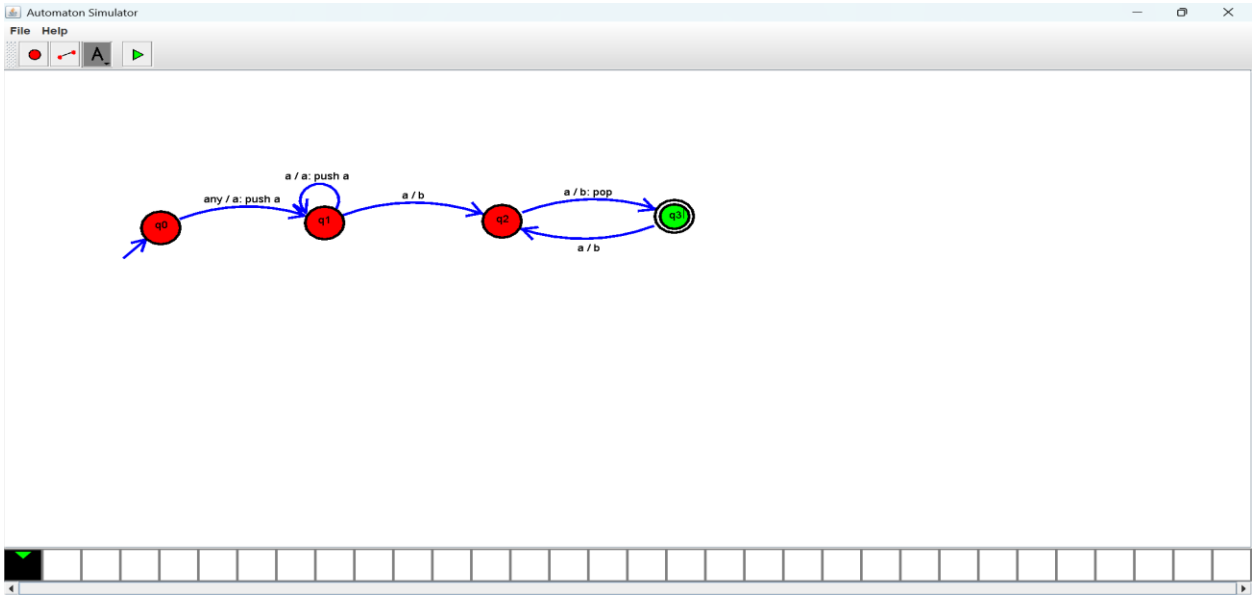
Exercise 13:

Output :



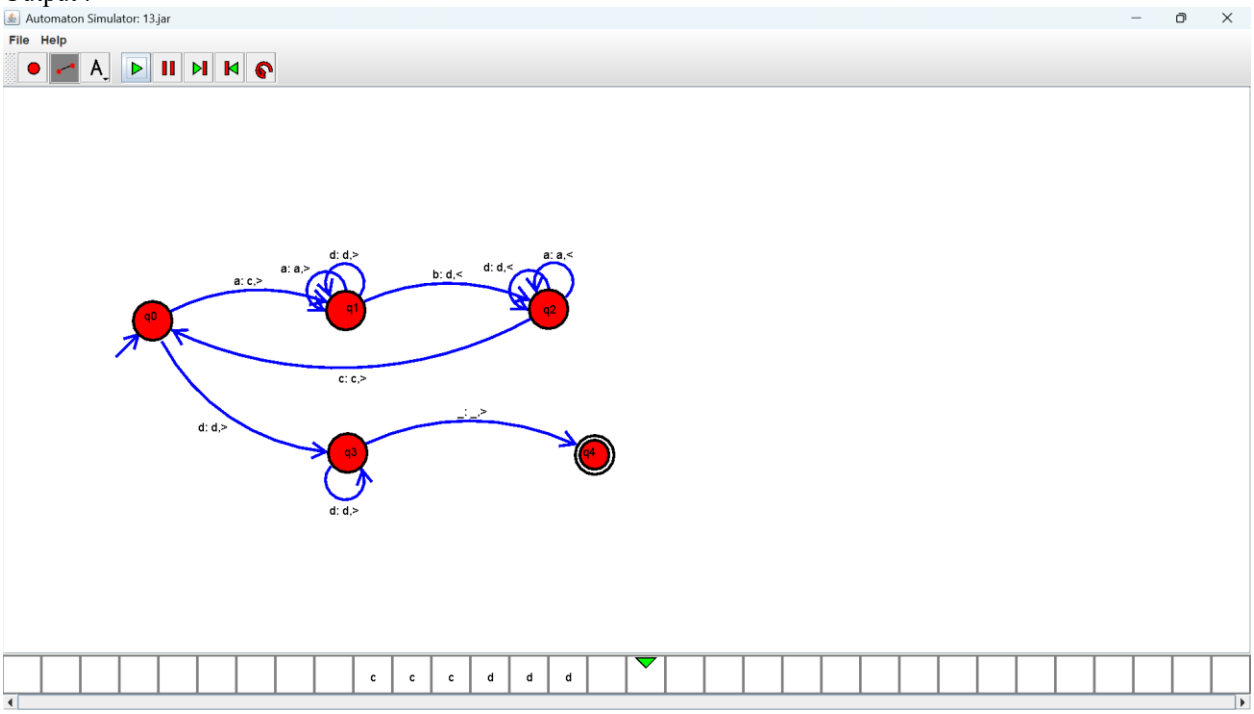

Exercise 14:

Output :



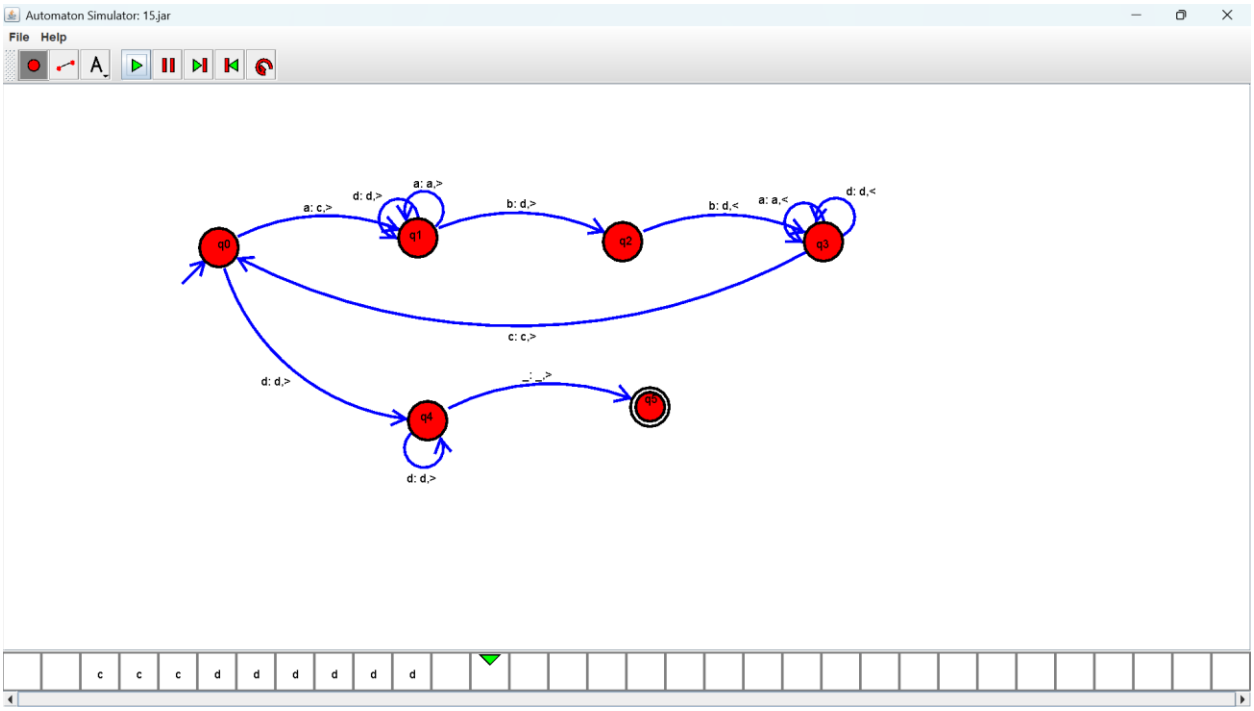

Exercise 15:

Output :



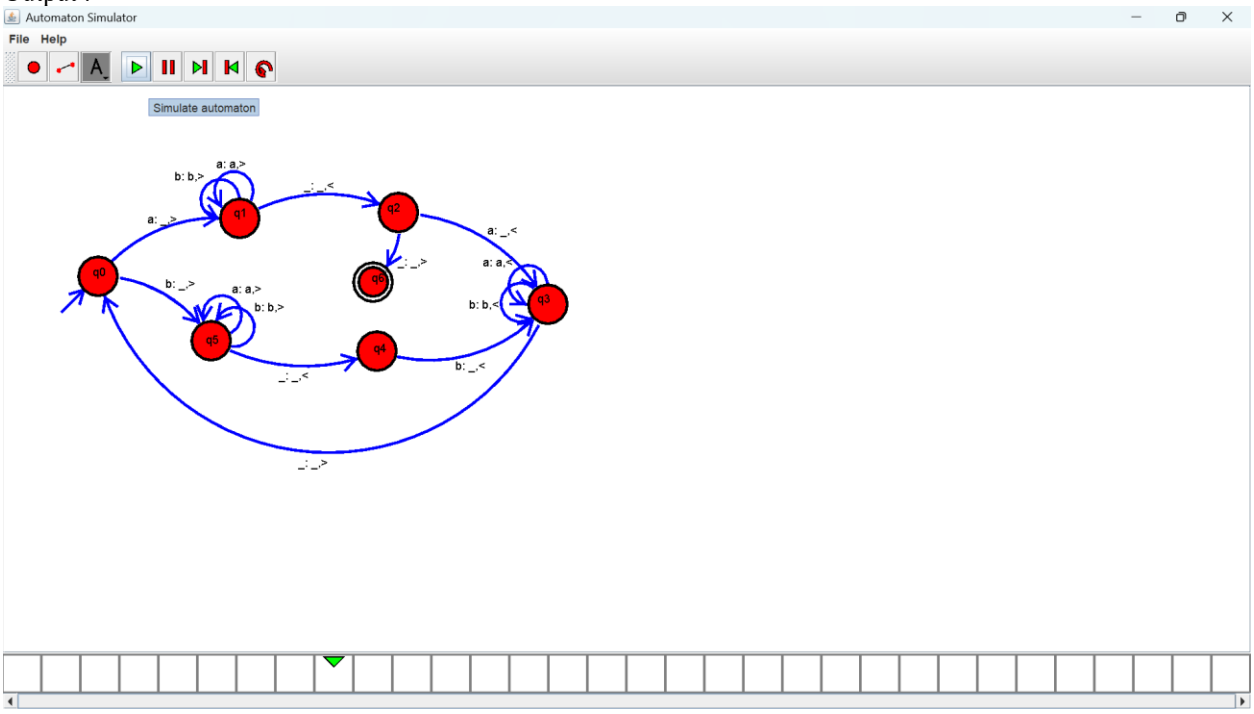

Exercise 16:

Output :




Exercise 17:

Output :

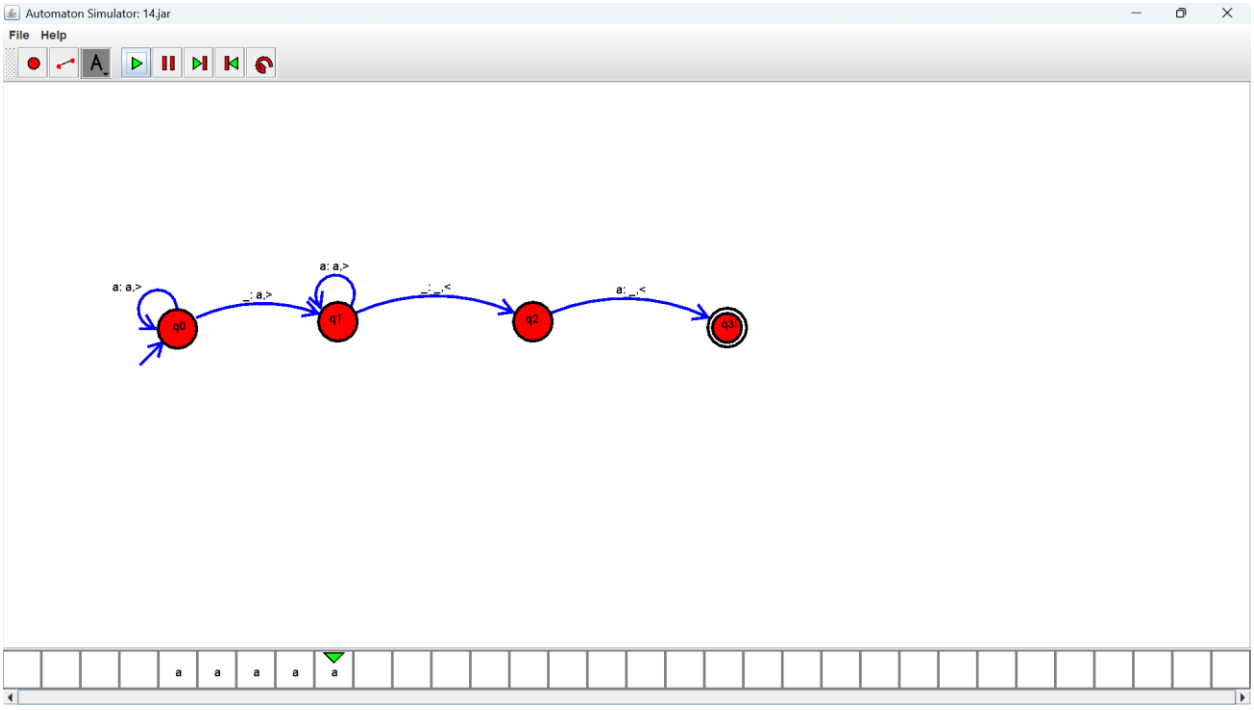



Exercise 18:

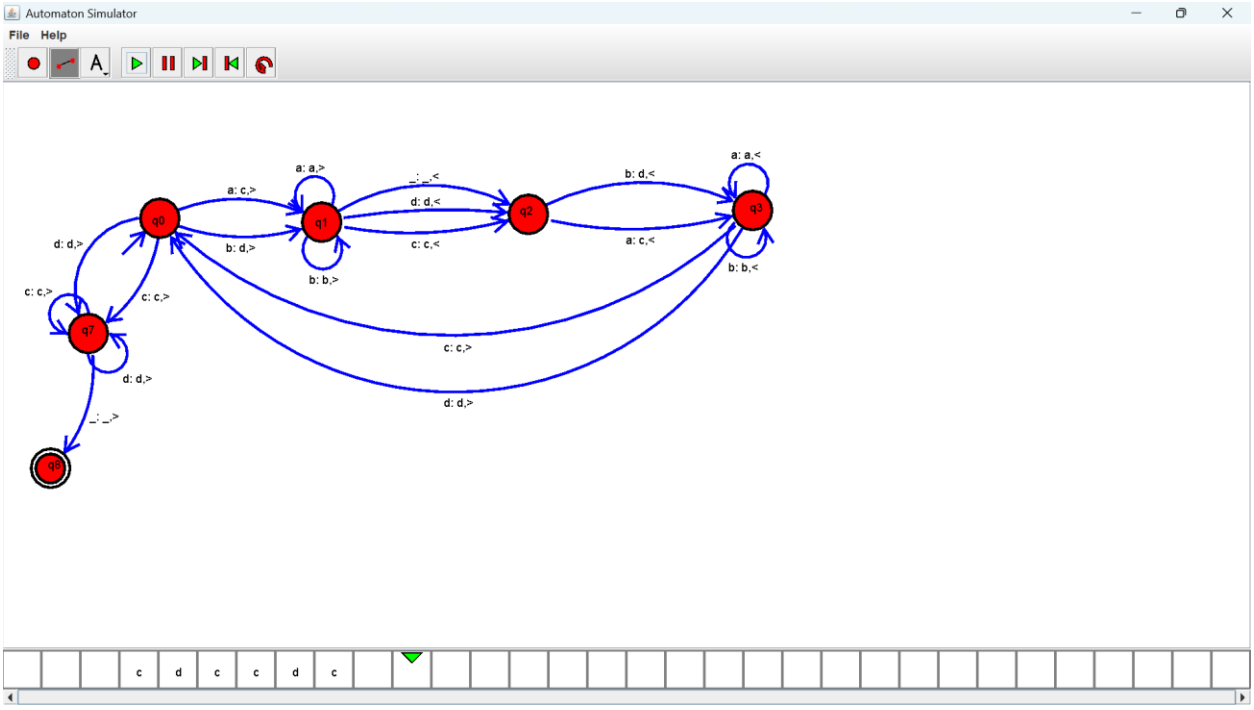
Output :

Exercise 19:

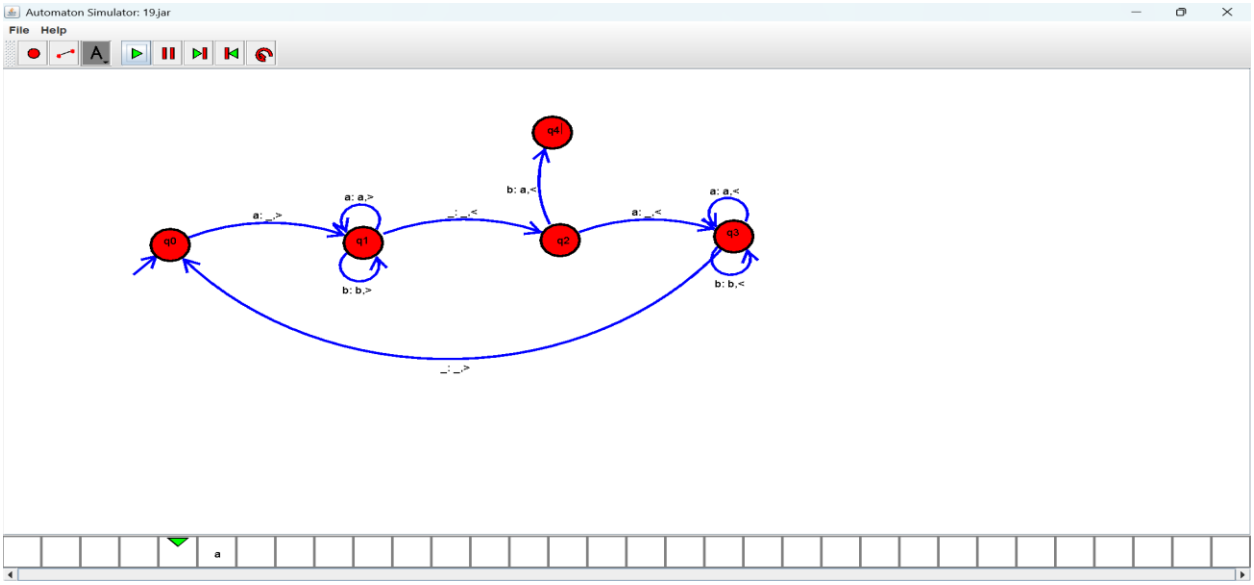
Output :




Exercise 20:

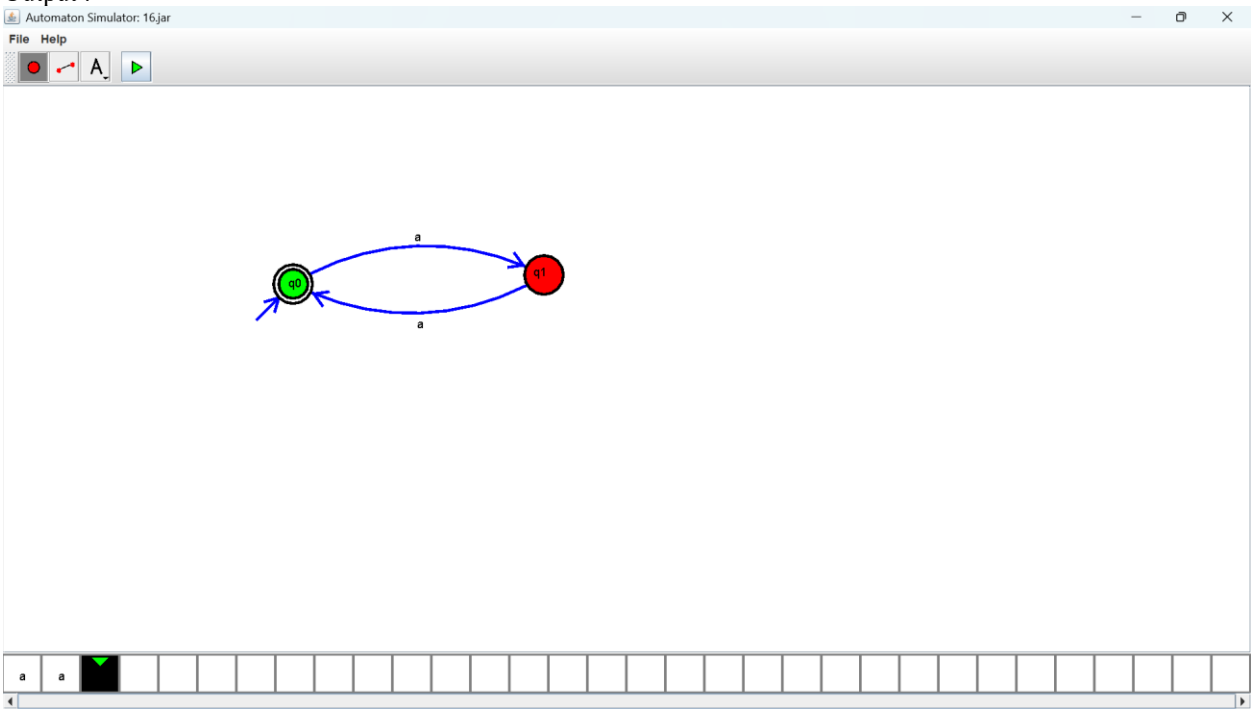


Output :



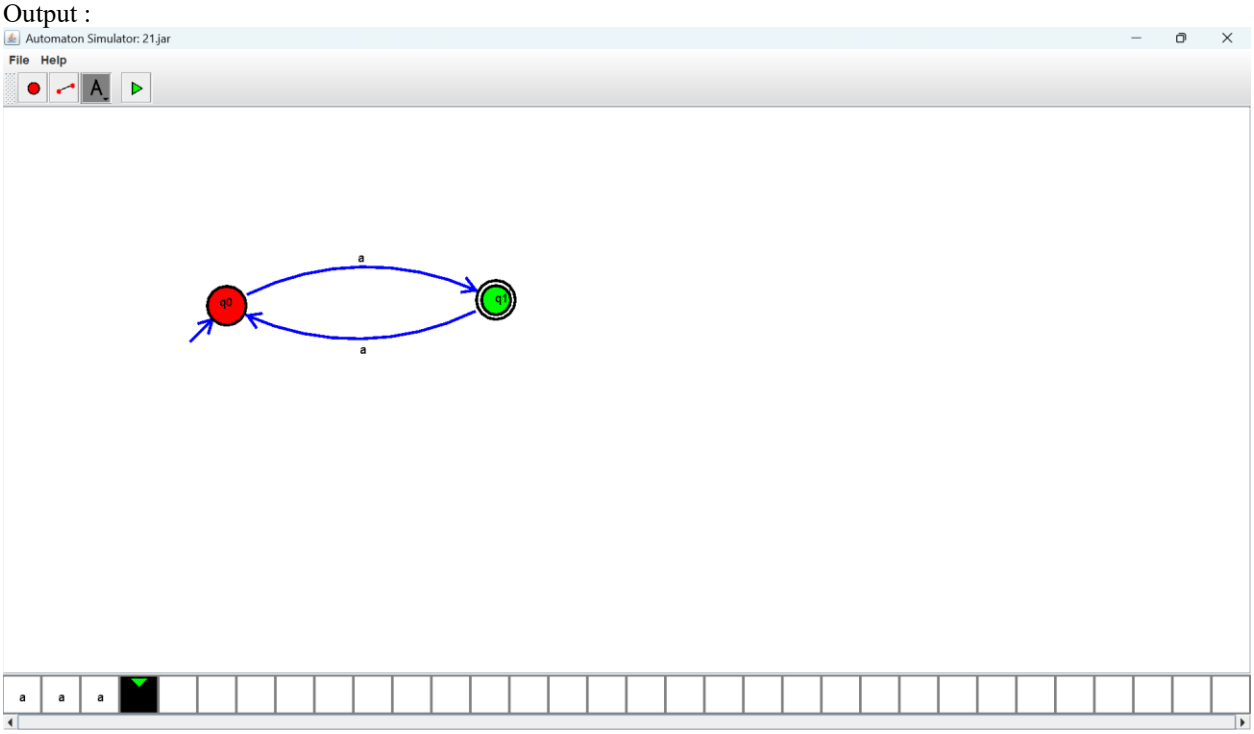

Exercise 21:

Output :



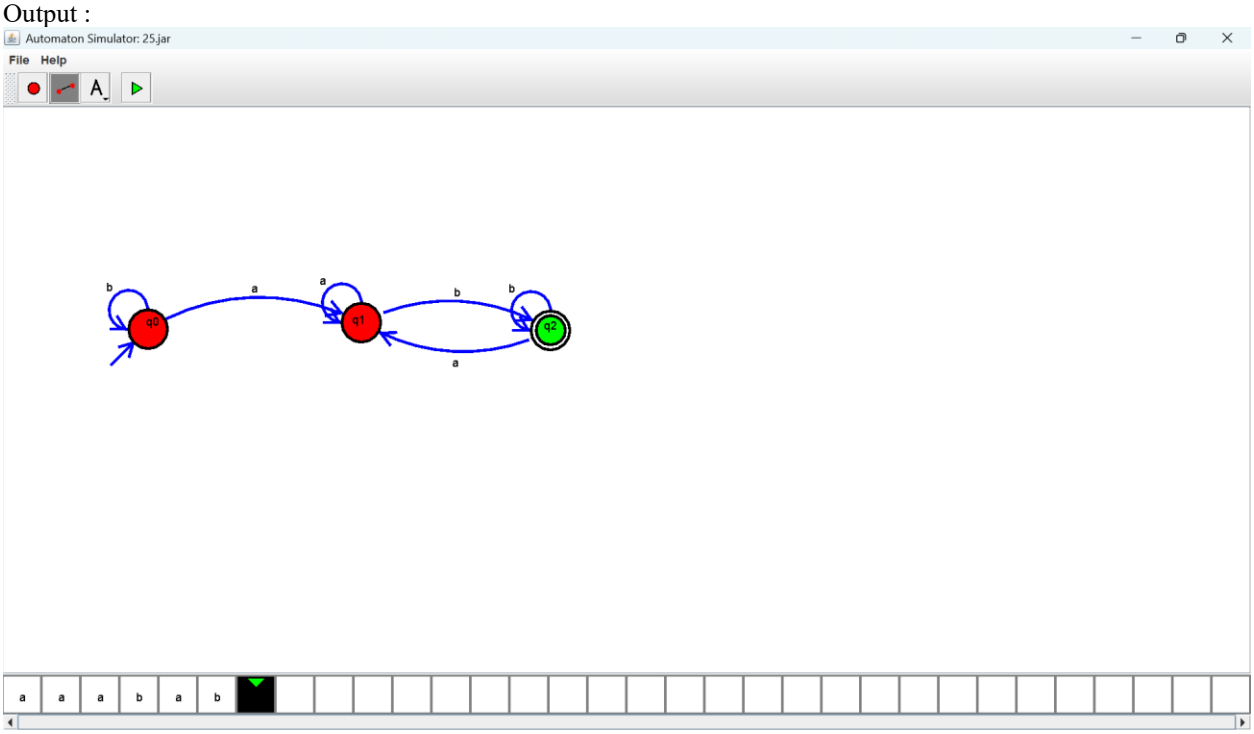
Exercise 22:

Output :



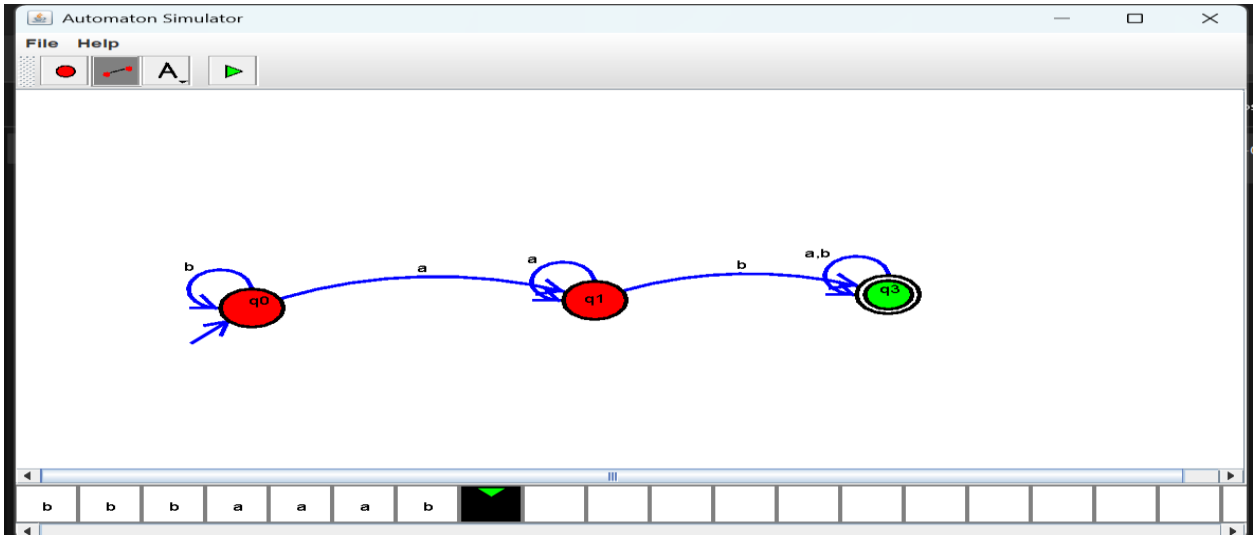

Exercise 23:

Output :



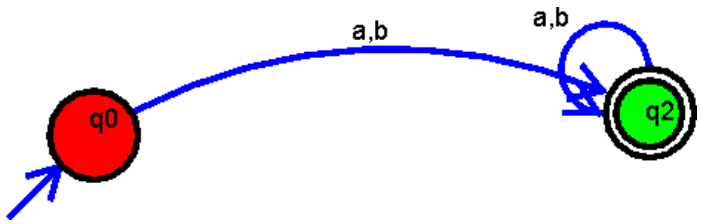
Exercise 24:

Output :



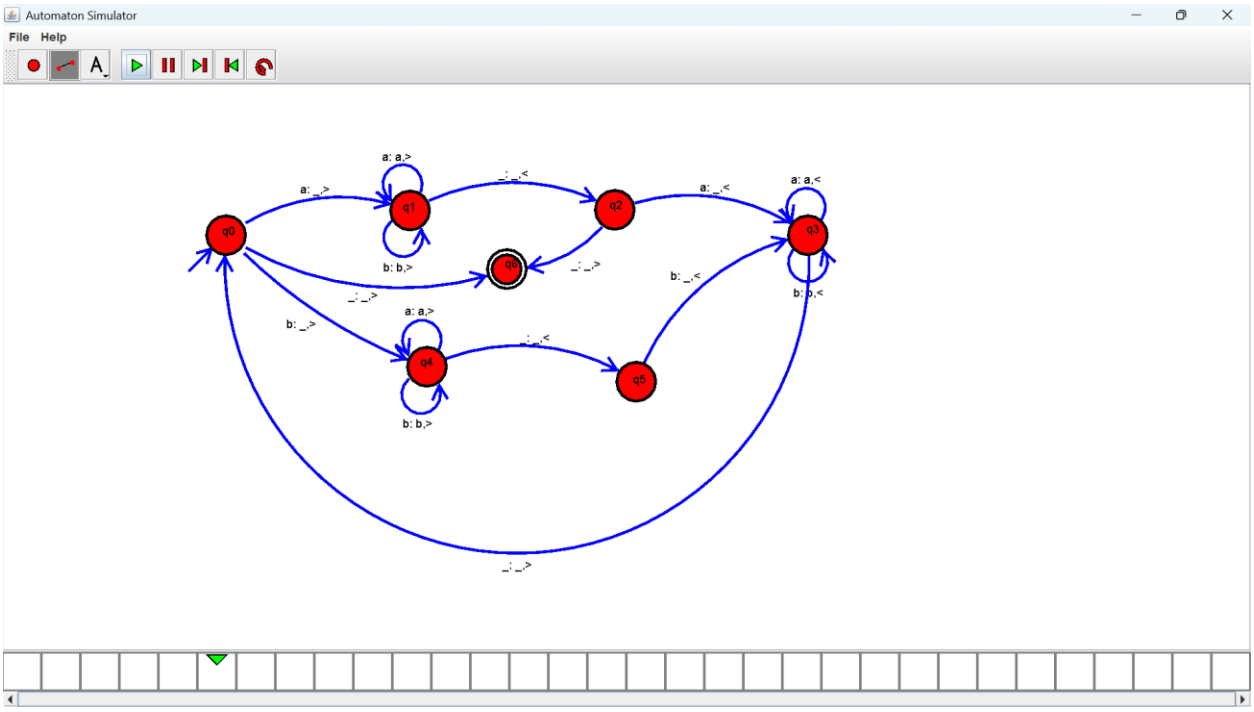

Exercise 25:

Output :



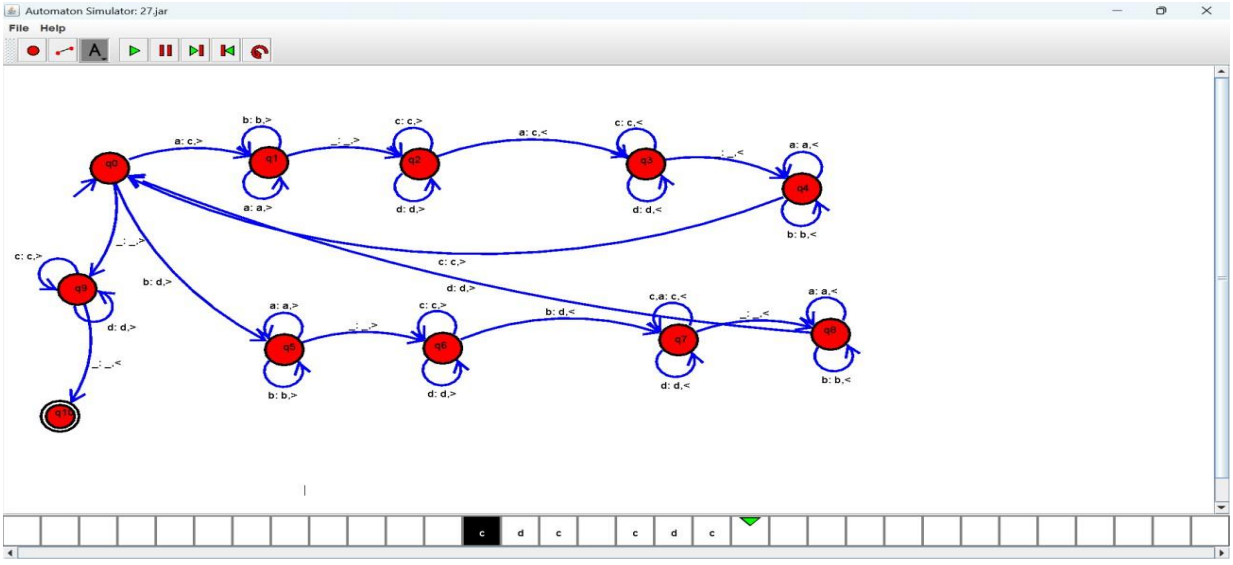

Exercise 26:

Output :



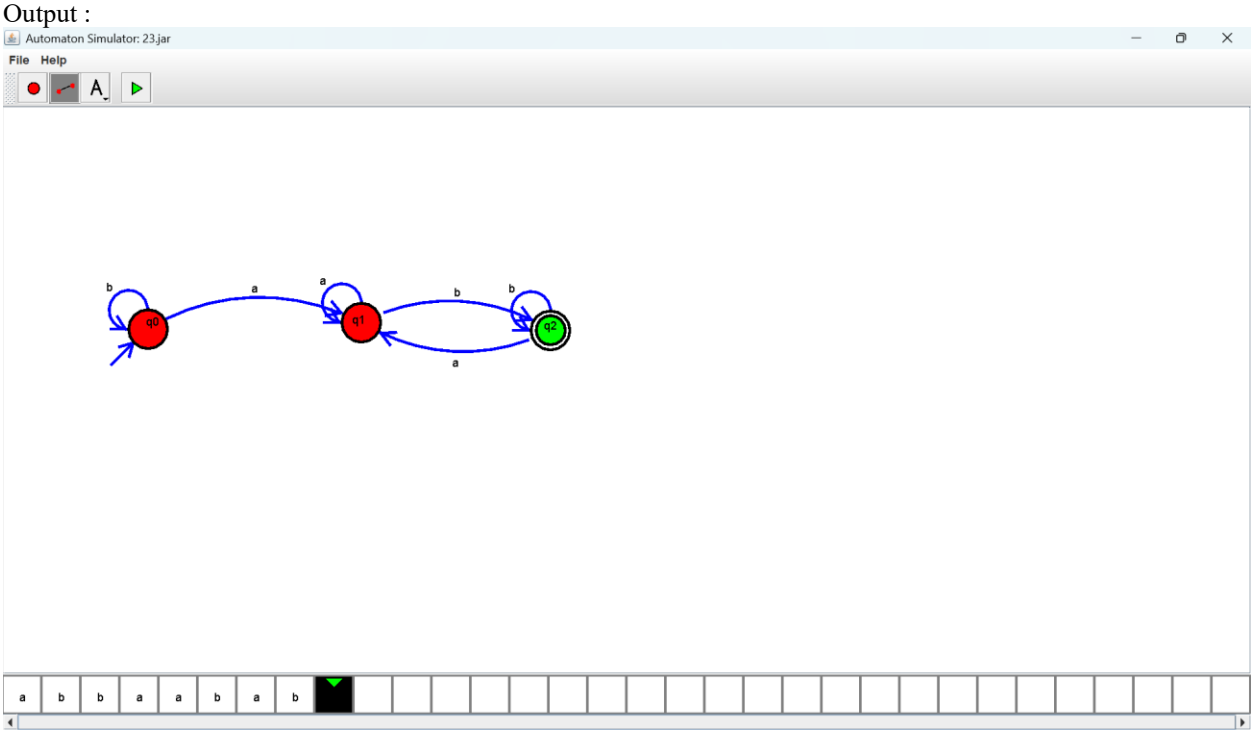

Exercise 27:

Output :



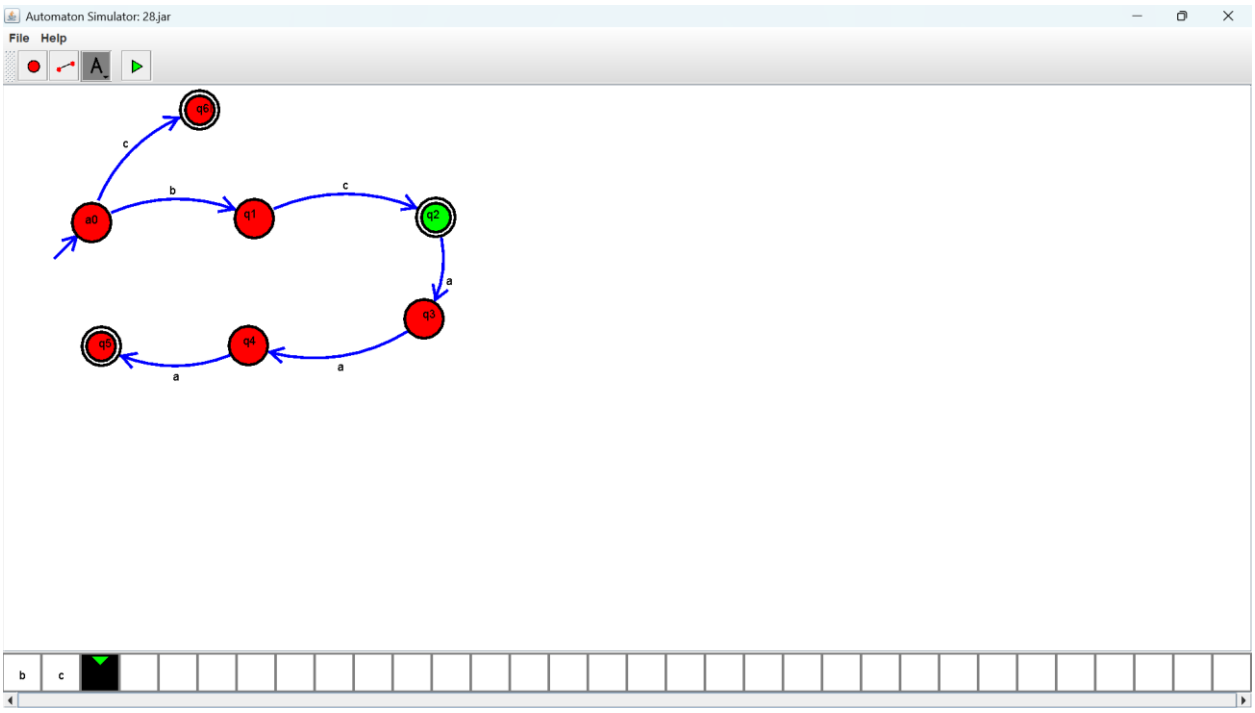
Exercise 28:

Output :



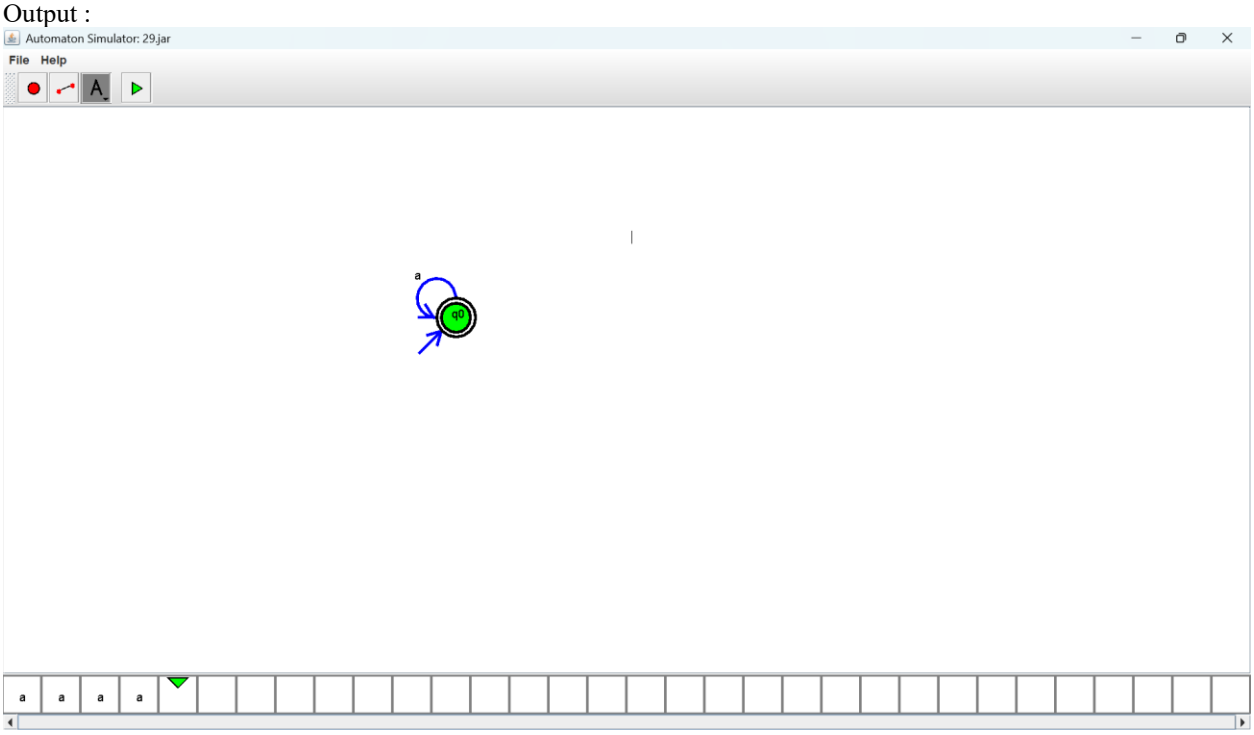

Exercise 29:

Output :



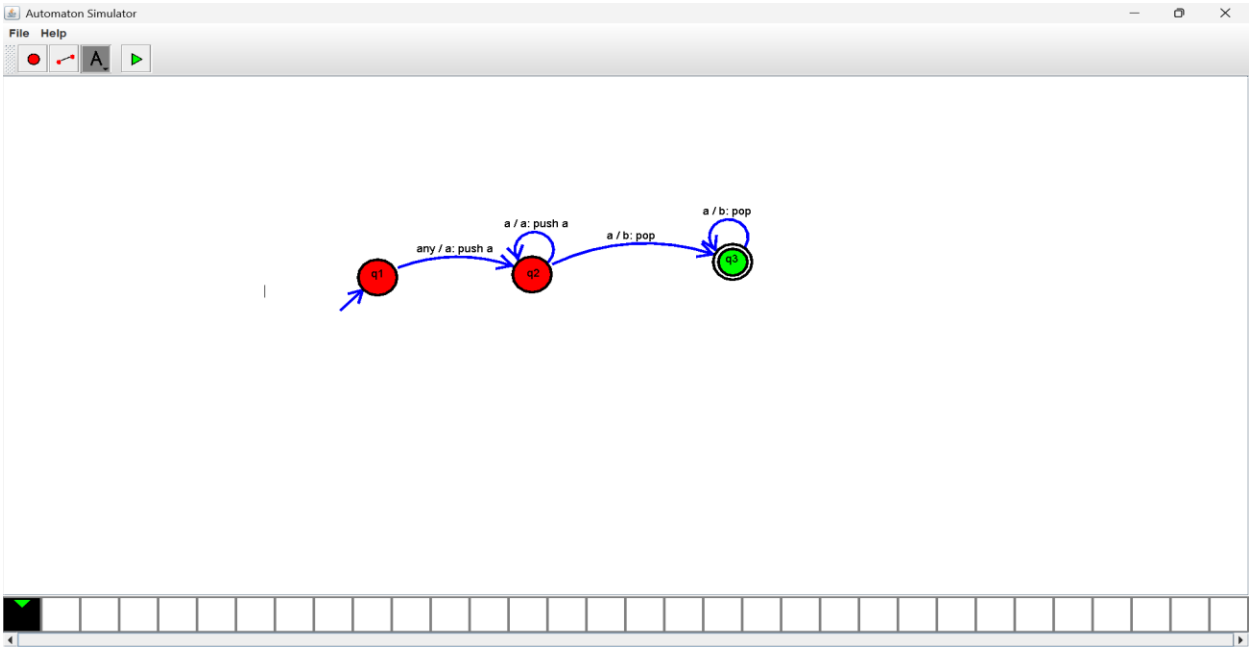
Exercise 30:

Output :



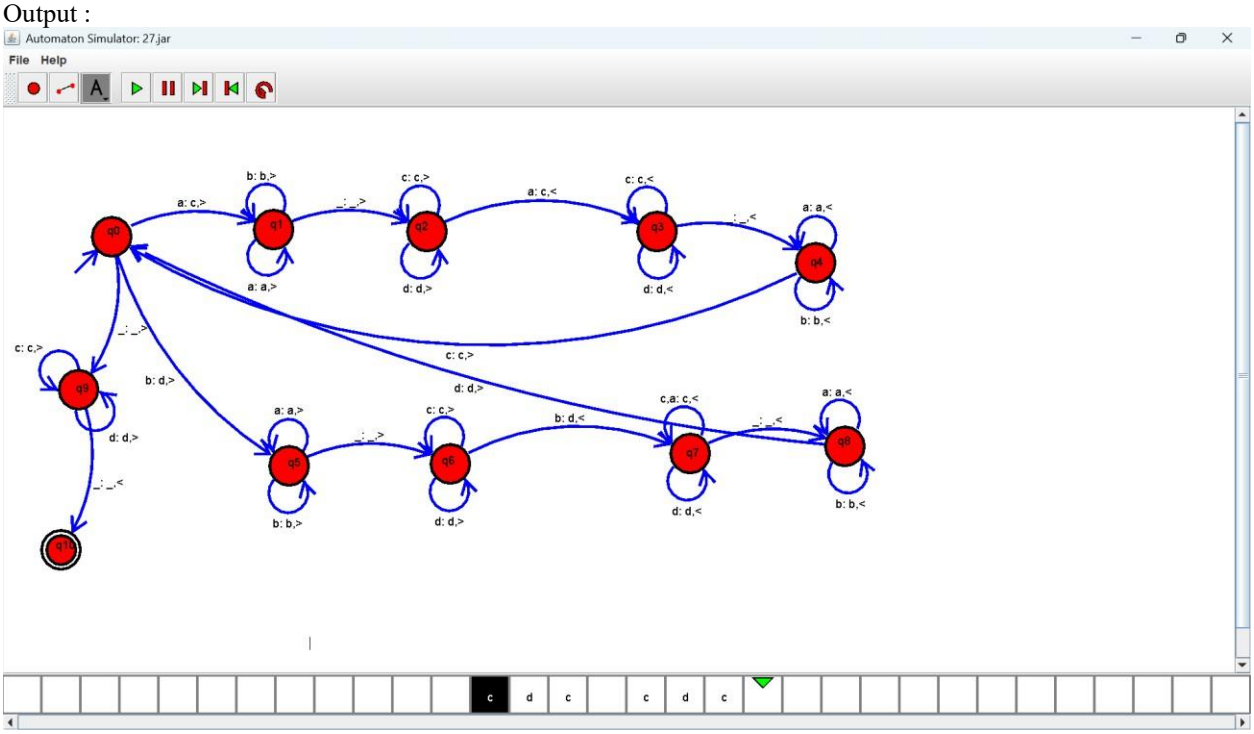

Exercise 31:

Output :



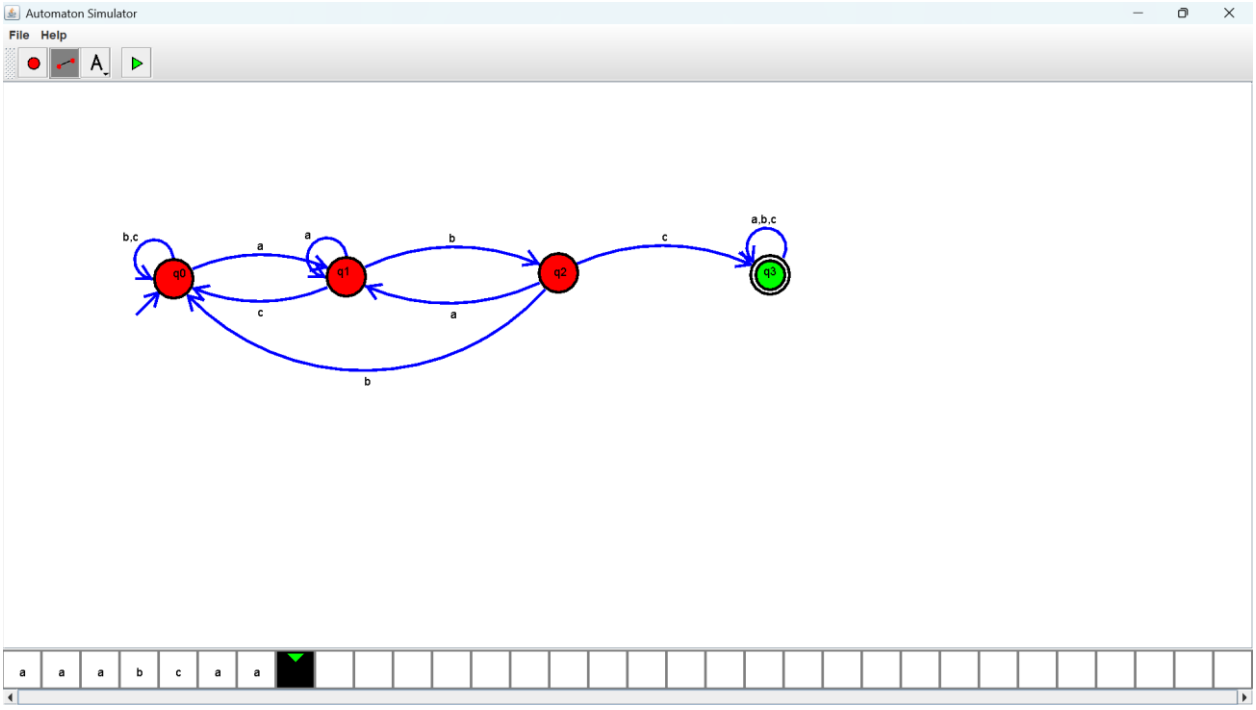
Exercise 32:

Output :



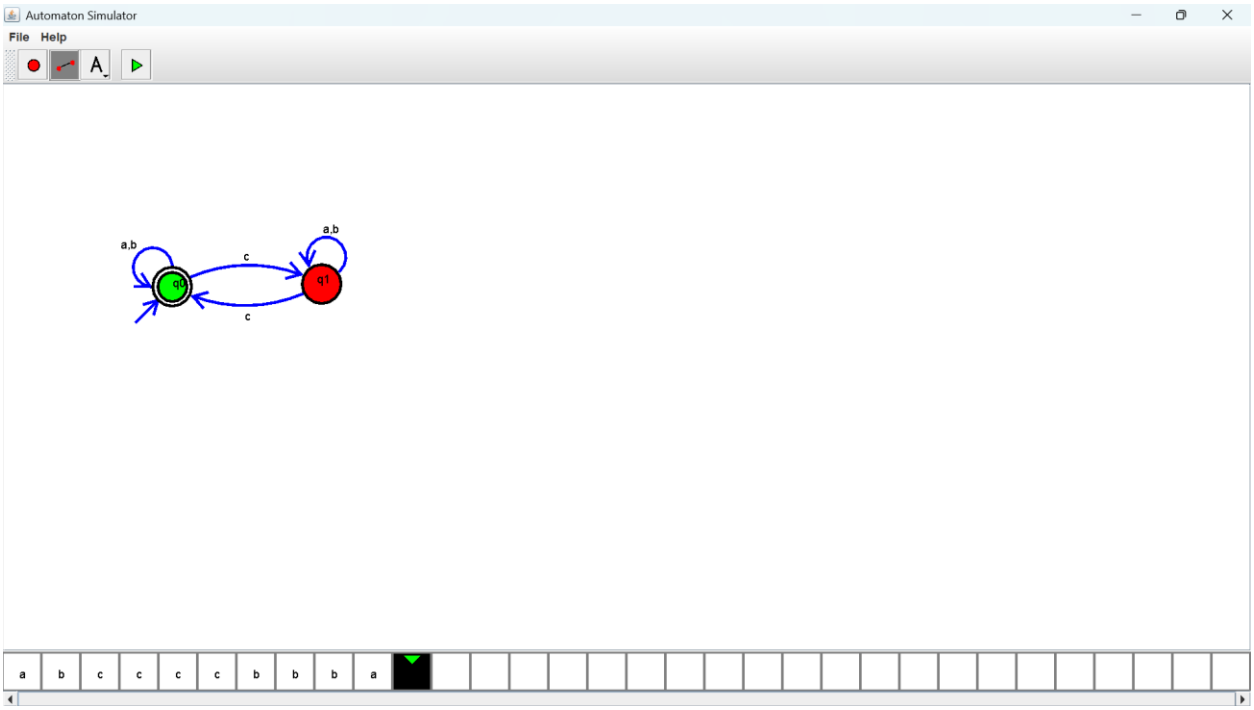

Exercise 33:

Output :



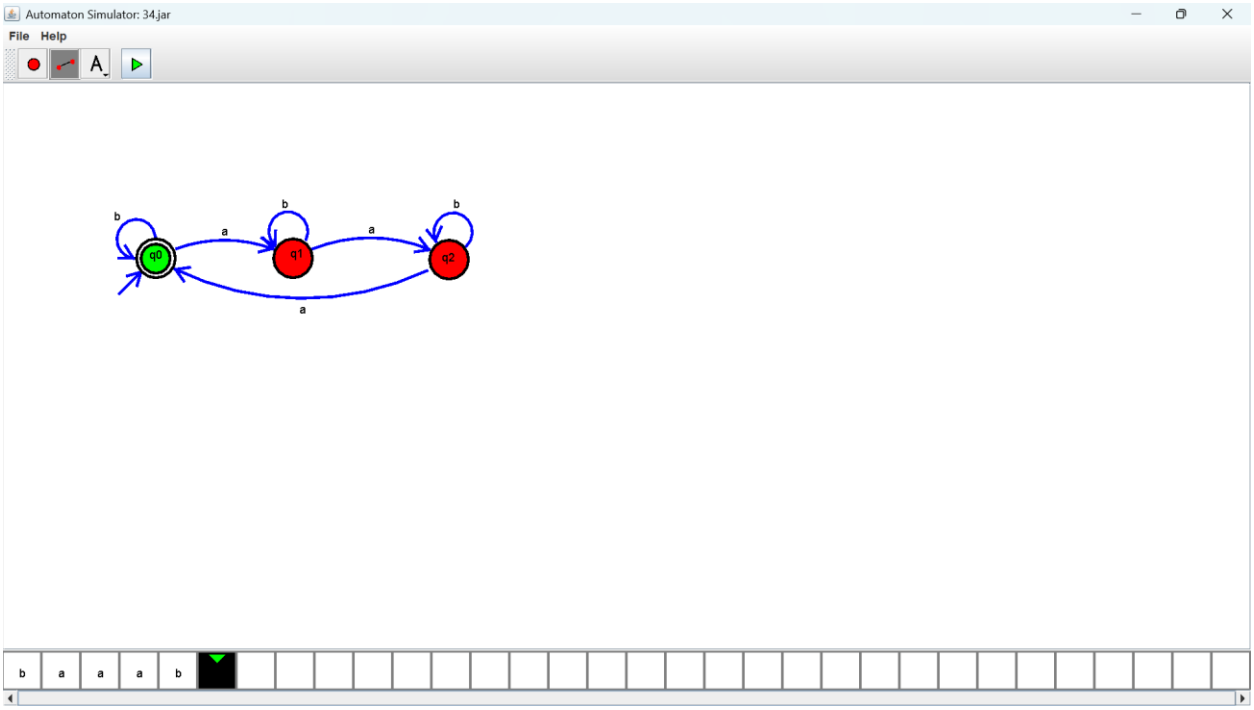
Exercise 34:

Output :



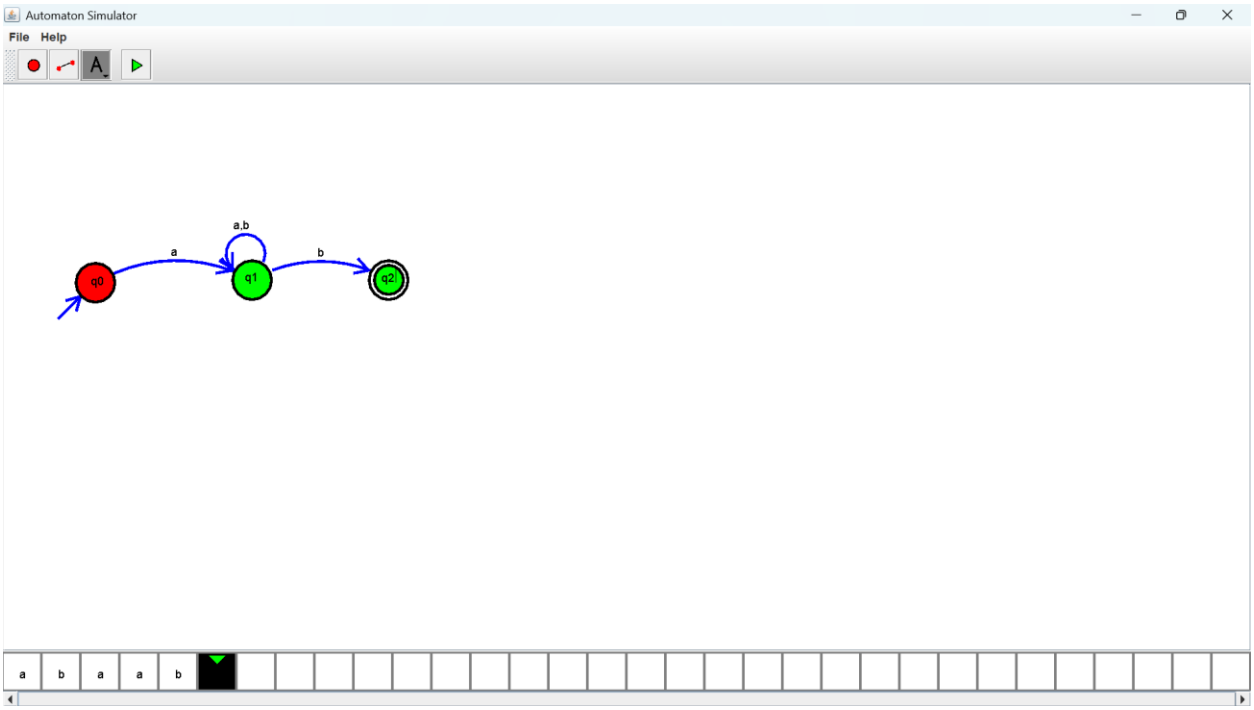

Exercise 35:

Output :



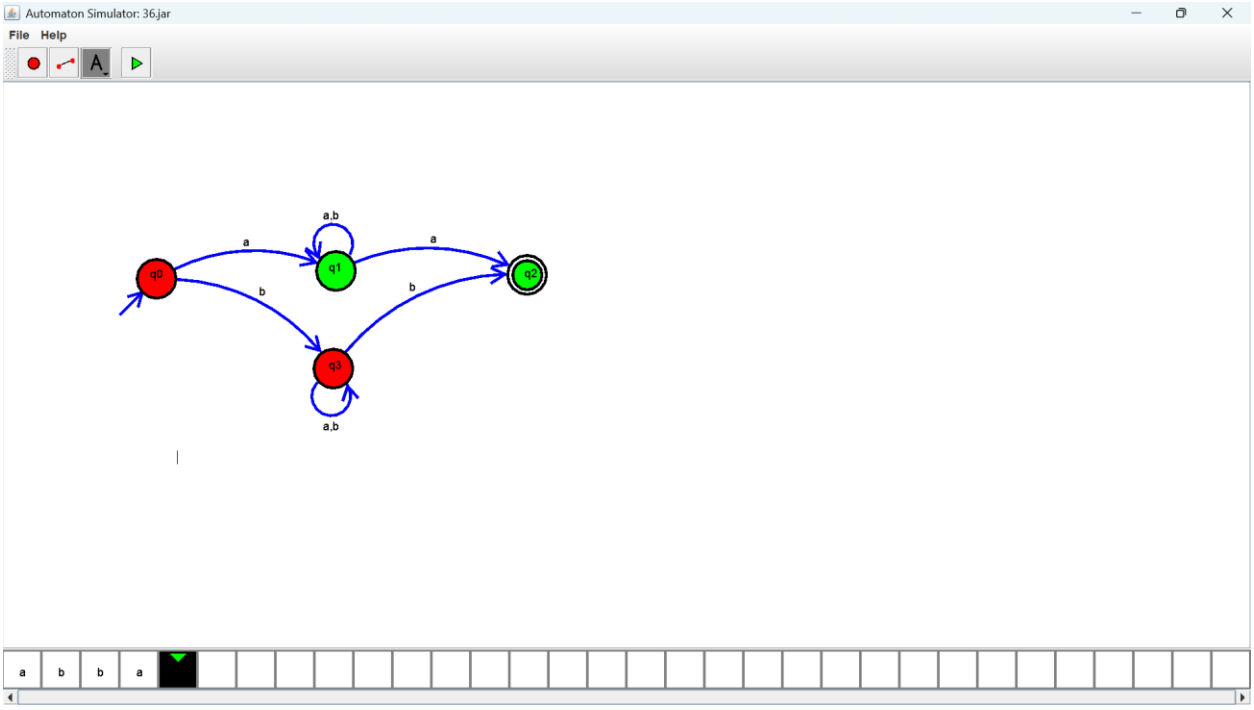
Exercise 36:

Output :



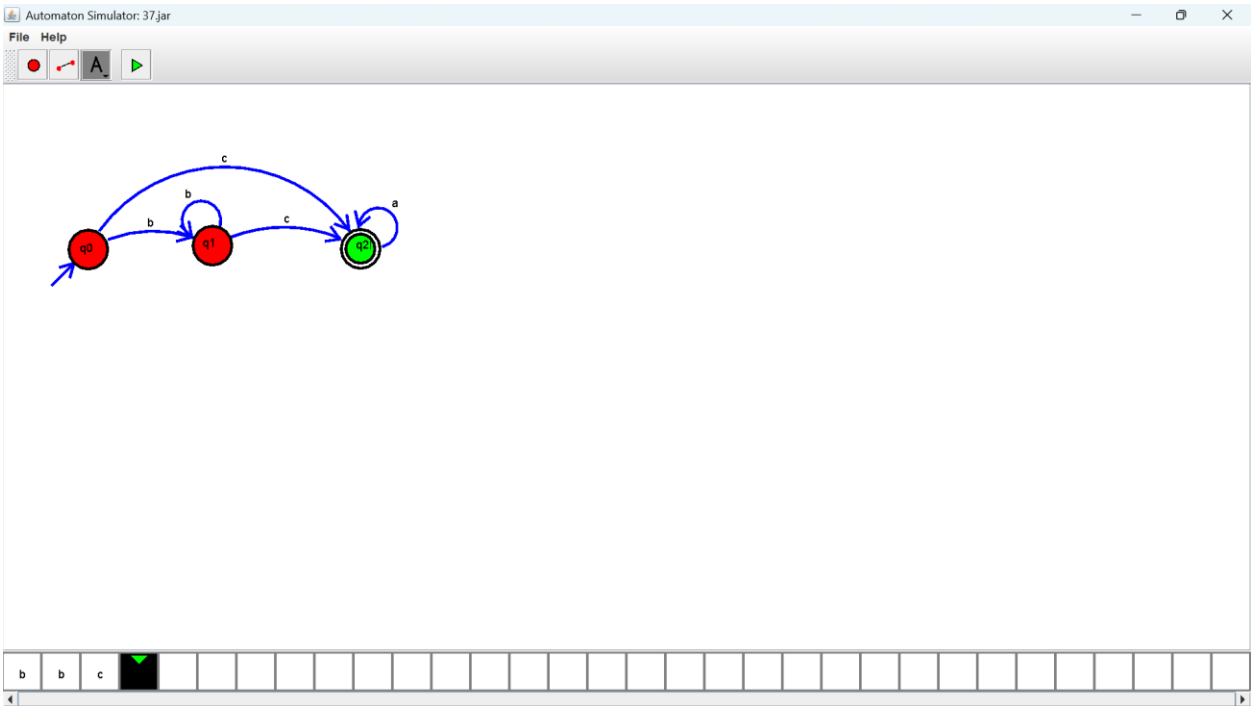

Exercise 37:

Output :



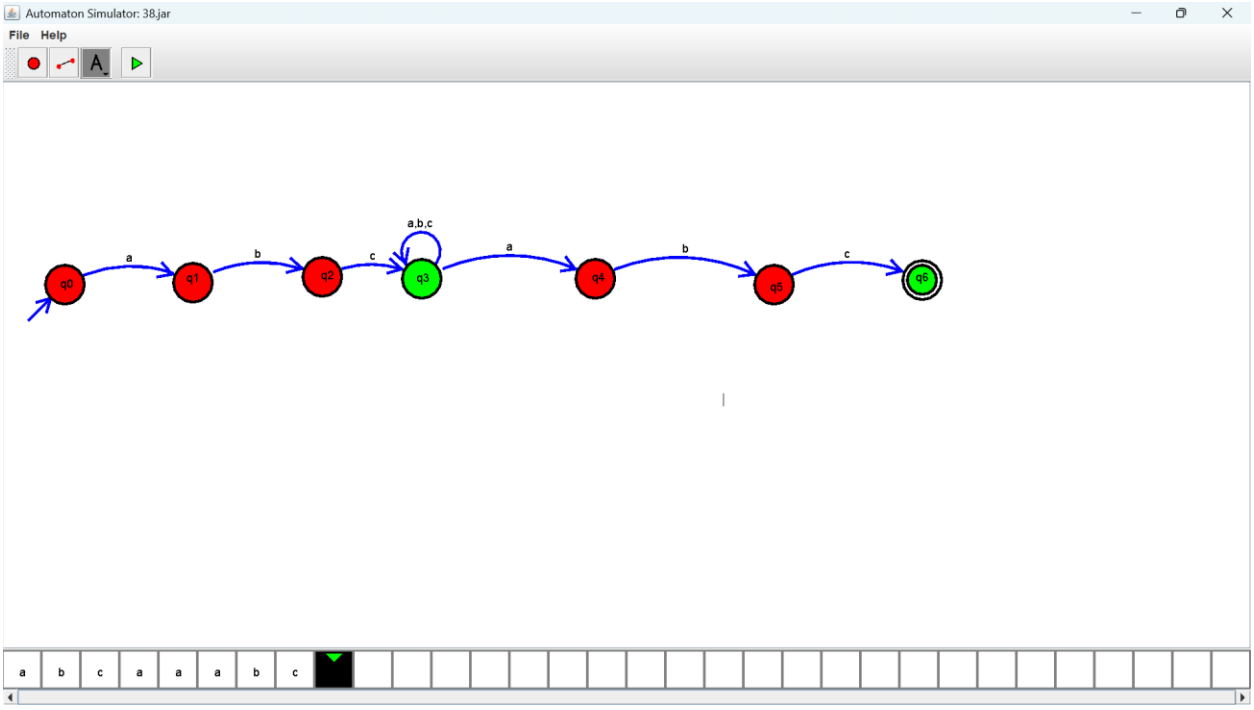
Exercise 38:

Output :




Exercise 39:

Output :



Exercise 40:

Output :